

pixel_battle Native Hi-Res Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: 讓打鬥畫面原生渲染成 1080x1920,消除目前 480x854 -> 1080x1920 的 2.25 倍上採樣,且打鬥行為完全不變。

Architecture: 物理與渲染分離。物理座標系一個字不動;新增 `scaled_pygame` shim 模組,在繪圖出口把座標、線寬、Surface 尺寸乘上縮放係數 S 。渲染層檔案頂端把 `import pygame` 換成 `shim`,呼叫點零修改。Surface 的 `blit` / `fill` / `subsurface` 透過 `pygame.Surface` 子類覆寫(已實測可行)。

Tech Stack: Python 3.10.11 / pygame 2.6.1 (SDL 2.28.4) / pytest / numpy 1.26.4 / scikit-image 0.19.3 (SSIM) / ffmpeg

Spec: `docs/superpowers/specs/2026-08-24-pixel-battle-native-hires-design.md`

Global Constraints

- 物理/邏輯層零改動。 `engine/physics.py` 、 `engine/battle.py` 、 `engine/character.py` 、 `engine/skill.py` 、 `engine/rng.py` 、 `engine/effects.py` 全部零 `pygame` import,不得引入 `shim`,不得修改。
- 座標常數不動。 `renderer.py` `WIDTH=480` / `HEIGHT=854`; `physics.py` `ARENA_LEFT=10` / `ARENA_RIGHT=470` / `GROUND_Y=700`。
- S 預設 2.25。 $S=1.0$ 必須讓所有輸出與改動前逐位元組相同。
- 主畫布固定 1080x1920。 `Surface((480, 854))` 特例映射到 `(1080, 1920)`,不用 `round(854*2.25)=1922`。超出的 1.5px 由 `pygame` 自然裁切;`GROUND_Y` 縮放後為 1575,距底部 345px,無可見元素受影響。
- 線寬與 `radius` 取 `max(1, round(v * S))`,避免細線取整後消失。但 `pygame` 中 `width=0` 代表實心填滿,必須原樣保留 0。
- 不改 `engine/` 的 `episode` 渲染路徑 (`animator/banner/charge_fx/cinematic/hud/impact_fx/particles/projectile/renderer`),不在 b01-b21 主線上。
- 不改 `episodes/` 、 `video/captions.py`。
- 測試環境: 測試檔開頭需 `os.environ.setdefault("SDL_VIDEODRIVER", "dummy")`。專案目錄下 `python3` 即 3.10.11,不需設 `PYENV_VERSION`(`worktree` 中才需要 `PYENV_VERSION=3.10.11`)。

- 既有 5 個失敗測試不列入驗收,它們在動手前即為紅燈(已於 5829f7a 實測確認):
`test_battle.py::test_ultimate_triggers_when_mp_full` 、
`test_battle.py::test_ultimate_locks_combat_during_playback` 、
`test_hitstop.py::test_basic_hit_causes_hitstop` 、
`test_hitstop.py::test_basic_non_crit_sets_hitstop_via_resolve` 、
`test_poses.py::test_all_poses_keep_feet_planted_and_in_frame` 。基準是 555 passed, 5 failed 。
 - 每個 task 結束都要 commit 。
-

Task 1: 建立 S=1.0 基準快照

必須在任何渲染代碼改動前完成。後續兩層驗證都以此為準。

Files: – Create: `tools/hires_baseline.py` – Create: `pixel_battle/output/hires_baseline/` (產物,加入 `.gitignore`) – Modify: `.gitignore`

Interfaces: – Produces: `tools/hires_baseline.py` 提供 `capture(script_path, out_dir, frame_indices) -> dict`,寫出 `events.json` (含 `events` / `event_video_ms` / `winner` / `n_frames`)與 `frame_{idx}.png` 。後續 Task 7 的比對工具讀這些檔案。

☐ Step 1: 寫基準擷取工具

關鍵設計: 不要自己重組渲染流程。 `render_script` 內含 arena 主題選擇、 timeline seed、 `start_mp/start_hp` 覆寫、 `end_hold_frames=120` 等組裝細節, 自行重寫必然與正式管線不一致,基準就失去意義。改為 patch 兩個名字後直接 呼叫 `render_script` 。

Create `tools/hires_baseline.py`:

```
"""Capture a render baseline: fight events + sampled frames.

Used to prove the native hi-res work changes nothing about the fight and
nothing about frame composition. Run BEFORE touching render code.

This reuses play_scripted.render_script rather than reassembling the render
itself – that function owns arena selection, timeline seeding and start
HP/MP overrides, and a hand-rolled copy would drift from the real pipeline.
"""

from __future__ import annotations
import json
import os
```

```

import sys
from pathlib import Path

os.environ.setdefault("SDL_VIDEODRIVER", "dummy")

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

import pygame # noqa: E402

# Frames sampled across the fight arc: open, melee, ranged standoff,
# special cast, ultimate, KO aftermath. Frame numbers at 60fps.
DEFAULT_FRAMES = (60, 600, 1200, 1800, 2400, 2900, 3200)

def capture(script_path: Path, out_dir: Path,
            frame_indices=DEFAULT_FRAMES) -> dict:
    """Render `script_path` via the real pipeline, recording events + frames."""
    import pixel_battle.rl.play_scripted as ps
    from pixel_battle.video.recorder import FrameRecorder

    out_dir = Path(out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)

    wanted = set(frame_indices)
    grabbed: dict[int, object] = {}
    captured: dict = {}

    class _TapRecorder(FrameRecorder):
        """Writes frames as usual, and snapshots the sampled indices."""

        def __init__(self, *a, **k):
            super().__init__(*a, **k)
            self._n = 0

        def write_frame(self, surface):
            if self._n in wanted:
                grabbed[self._n] = surface.copy()

```

```

        self._n += 1
        return super().write_frame(surface)

orig_recorder = ps.FrameRecorder
orig_render_fight = ps._render_fight

def _tap_render_fight(*a, **k):
    result = orig_render_fight(*a, **k)
    captured.update(result)
    return result

ps.FrameRecorder = _TapRecorder
ps._render_fight = _tap_render_fight
try:
    ps.render_script(Path(script_path), out_dir)
finally:
    ps.FrameRecorder = orig_recorder
    ps._render_fight = orig_render_fight

for idx, surf in grabbed.items():
    pygame.image.save(surf, str(out_dir / f"frame_{idx:05d}.png"))

sample = next(iter(grabbed.values()), None)
payload = {
    "n_frames": captured.get("n_frames"),
    "winner": captured.get("winner"),
    "terminated": captured.get("terminated"),
    "events": captured.get("events", []),
    "event_video_ms": captured.get("event_video_ms", {}),
    "canvas": list(sample.get_size()) if sample is not None else None,
    "frames_captured": sorted(grabbed),
}
(out_dir / "events.json").write_text(
    json.dumps(payload, indent=2, ensure_ascii=False, default=str),
    encoding="utf-8")
return payload

```

```

if __name__ == "__main__":

```

```

script = Path(sys.argv[1]) if len(sys.argv) > 1 else \
    ROOT / "pixel_battle/data/scripts/b01_lumen_jugg.yaml"
out = Path(sys.argv[2]) if len(sys.argv) > 2 else \
    ROOT / "pixel_battle/output/hires_baseline"
p = capture(script, out)
print(f"baseline: {p['n_frames']} frames, winner={p['winner']}, "
      f"{len(p['events'])} events, canvas={p['canvas']}, "
      f"frames={p['frames_captured']}")

```

已確認的 API 事實(不需再查): – `FrameRecorder` 只有 `start()` / `write_frame()` / `stop()` ,沒有 `close()` — `render_script` 已負責 start/stop,本工具不碰。 – `ps._render_fight` 是 `play_scripted` 以 `from ... import` 綁進來的名字, patch 必須打在 `ps` 上,不是 `play` 上。 – `ps.FrameRecorder` 同理。

☐ Step 2: 產生兩份基準(b01 近戰、b12 遠距)

遠距對峙會走到與近戰不同的繪圖路徑(投射物、拉開的鏡頭距離),只驗一部 會漏掉一整類漏縮放。兩份基準都必須在改動前產生。

Run:

```

python3 tools/hires_baseline.py \
    pixel_battle/data/scripts/b01_lumen_jugg.yaml \
    pixel_battle/output/hires_baseline/b01
python3 tools/hires_baseline.py \
    pixel_battle/data/scripts/b12_shade_deadeye.yaml \
    pixel_battle/output/hires_baseline/b12

```

Expected: 各印出 `baseline: <N> frames, winner=..., <M> events, canvas=[480, 854], frames=[...]`

☐ Step 3: 確認基準幀尺寸為 480x854

Run:

```

python3 -c "
from PIL import Image
for b in ('b01', 'b12'):
    im = Image.open(f'pixel_battle/output/hires_baseline/{b}/frame_00600.png')

```

```
print(b, im.size)

"
```

Expected: 兩行都是 (480, 854)

☐ Step 4: 把基準產物排除於 git 之外

Append to `.gitignore`:

```
# Native hi-res render baseline (regenerable via tools/hires_baseline.py)
pixel_battle/output/hires_baseline/
```

☐ Step 5: Commit

```
git add tools/hires_baseline.py .gitignore
git commit -m "test(pixel-battle): 建立 hi-res 基準擷取工具 - events + 抽樣幀"
```

Task 2: shim 骨架與 draw.line

Files: – Create: `pixel_battle/rl/scaled_pygame.py` – Test:

`pixel_battle/tests/test_scaled_pygame.py`

Interfaces: – Produces: 模組級 `S: float`、`set_scale(s: float) -> None`、`CANVAS: tuple[int,int]`、`CANVAS_SCALED: tuple[int,int]`、`_pt(p) -> tuple[float,float]`、`_len(v) -> int`、`draw.line(...)`,以及 `__getattr__` 轉發。後續 Task 3-5 在同一模組上擴充。

☐ Step 1: 寫失敗測試

Create `pixel_battle/tests/test_scaled_pygame.py`:

```
"""scaled_pygame shim – coordinate/size scaling at the drawing boundary."""
import os
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")

def _fresh(scale):
    """Import the shim and set its scale."""
    from pixel_battle.rl import scaled_pygame as sp
    sp.set_scale(scale)
```

```
return sp
```

```
def test_scale_defaults_to_2_25():  
    from pixel_battle.rl import scaled_pygame as sp  
    sp.set_scale(2.25)  
    assert sp.S == 2.25
```

```
def test_unknown_attributes_forward_to_real_pygame():  
    import pygame  
    sp = _fresh(2.25)  
    assert sp.SRCALPHA == pygame.SRCALPHA  
    assert sp.BLEND_RGB_ADD == pygame.BLEND_RGB_ADD
```

```
def test_length_scaling_never_vanishes():  
    sp = _fresh(2.25)  
    assert sp._len(1) == 2          # round(2.25) == 2  
    assert sp._len(0.1) >= 1        # a hairline must stay visible  
    assert sp._len(0) == 0          # 0 means "filled" in pygame – preserve it
```

```
def test_draw_line_scales_endpoints():  
    import pygame  
    sp = _fresh(2.0)  
    surf = pygame.Surface((100, 100))  
    sp.draw.line(surf, (255, 0, 0), (5, 5), (5, 20), 1)  
    # at S=2.0 the line runs from (10,10) to (10,40)  
    assert surf.get_at((10, 12))[:3] == (255, 0, 0)  
    assert surf.get_at((5, 6))[:3] != (255, 0, 0)
```

```
def test_scale_one_is_identity():  
    import pygame  
    sp = _fresh(1.0)  
    a = pygame.Surface((60, 60))  
    b = pygame.Surface((60, 60))  
    sp.draw.line(a, (0, 255, 0), (3, 4), (40, 44), 3)
```

```
pygame.draw.line(b, (0, 255, 0), (3, 4), (40, 44), 3)
assert pygame.image.tostring(a, "RGB") == pygame.image.tostring(b, "RGB")
```

☐ Step 2: 執行測試確認失敗

Run: `python3 -m pytest pixel_battle/tests/test_scaled_pygame.py -q` Expected: FAIL —
`ModuleNotFoundError: No module named 'pixel_battle.rl.scaled_pygame'`

☐ Step 3: 寫最小實作

Create `pixel_battle/rl/scaled_pygame.py`:

```
"""Drop-in pygame replacement that scales drawing to a larger canvas.
```

WHY THIS EXISTS

The fight simulates in a 480x854 coordinate system. Rendering natively at 1080x1920 used to mean multiplying every spatial constant in the engine – and missing one would silently change fight outcomes. Instead, physics is left completely alone and only the DRAWING boundary is scaled.

HOW TO USE IT

In a render module, replace the pygame import:

```
from pixel_battle.rl import scaled_pygame as pygame
```

Call sites stay untouched. This IS an implicit substitution: if you are debugging a render module and `pygame` behaves oddly, check its import.

WHAT IS NOT SCALED

Physics and game logic never import this module. See the plan/spec at docs/superpowers/specs/2026-08-24-pixel-battle-native-hires-design.md

```
"""
```

```
from __future__ import annotations
```

```
import pygame as _pg
```

```
# The simulation's coordinate system, and the canvas it maps onto.
```

```
CANVAS = (480, 854)
```

```
CANVAS_SCALED = (1080, 1920)
```



```
S: float = 2.25
```

```
def set_scale(s: float) -> None:
    """Set the global draw scale. 1.0 reproduces pre-hi-res output exactly."""
    global S
    S = float(s)
```

```
def _pt(p):
    """Scale a point. Accepts any (x, y) sequence."""
    return (p[0] * S, p[1] * S)
```

```
def _pts(seq):
    """Scale a sequence of points."""
    return [_pt(p) for p in seq]
```

```
def _len(v):
    """Scale a length (line width, radius).

    pygame treats width=0 as "filled", so 0 must survive untouched. Any
    positive length floors at 1 so hairlines never round away to nothing.
    """

    if v is None:
        return None

    if v <= 0:
        return v

    return max(1, round(v * S))
```

```
class _Draw:
    """Scaled counterparts of the pygame.draw functions we use."""

    @staticmethod
    def line(surface, color, start_pos, end_pos, width=1):
        return _pg.draw.line(surface, color, _pt(start_pos), _pt(end_pos),
                               _len(width))
```

```
draw = _Draw()
```

```
def __getattr__(name):  
    """Forward anything we do not override to the real pygame."""  
    return getattr(_pg, name)
```

☐ Step 4: 執行測試確認通過

Run: `python3 -m pytest pixel_battle/tests/test_scaled_pygame.py -q` Expected: PASS (5 passed)

☐ Step 5: Commit

```
git add pixel_battle/rl/scaled_pygame.py pixel_battle/tests/test_scaled_pygame.py  
git commit -m "feat(pixel-battle): scaled_pygame shim 骨架 - 座標/長度縮放 + draw.line"
```

Task 3: 補齊 draw API 與 gfxdraw

主線用量: `line` 139、`circle` 92、`polygon` 46、`rect` 33、`lines` 7、`ellipse` 7、`gfxdraw.aacircle` 2、`gfxdraw.filled_circle` 1。

Files: – Modify: `pixel_battle/rl/scaled_pygame.py` – Test: `pixel_battle/tests/test_scaled_pygame.py`

Interfaces: – Consumes: Task 2 的 `_pt` / `_pts` / `_len` / `draw`。 – Produces: `draw.circle/polygon/rect/lines/ellipse`、`gfxdraw.aacircle/filled_circle`、`_rect(r) -> pygame.Rect`。

☐ Step 1: 寫失敗測試

Append to `pixel_battle/tests/test_scaled_pygame.py`:

```
def test_draw_circle_scales_center_and_radius():  
    import pygame  
    sp = _fresh(2.0)  
    surf = pygame.Surface((100, 100))  
    sp.draw.circle(surf, (0, 0, 255), (10, 10), 4)
```

```
# centre moves to (20,20); radius becomes 8
assert surf.get_at((20, 20))[:3] == (0, 0, 255)
assert surf.get_at((20, 26))[:3] == (0, 0, 255) # inside r=8
assert surf.get_at((20, 32))[:3] != (0, 0, 255) # outside
```

```
def test_draw_circle_preserves_filled_width_zero():
    import pygame
    sp = _fresh(2.0)
    surf = pygame.Surface((100, 100))
    sp.draw.circle(surf, (255, 255, 0), (25, 25), 10, 0)
    assert surf.get_at((25, 25))[:3] == (255, 255, 0) # filled, not a ring
```

```
def test_draw_rect_scales_rect():
    import pygame
    sp = _fresh(2.0)
    surf = pygame.Surface((100, 100))
    sp.draw.rect(surf, (255, 0, 255), pygame.Rect(5, 5, 10, 10))
    assert surf.get_at((12, 12))[:3] == (255, 0, 255) # inside 10,10..30,30
    assert surf.get_at((8, 8))[:3] != (255, 0, 255)
```

```
def test_draw_polygon_scales_points():
    import pygame
    sp = _fresh(2.0)
    surf = pygame.Surface((100, 100))
    sp.draw.polygon(surf, (0, 255, 255), [(5, 5), (20, 5), (20, 20)])
    assert surf.get_at((30, 20))[:3] == (0, 255, 255)
```

```
def test_gfxdraw_scales():
    import pygame
    sp = _fresh(2.0)
    surf = pygame.Surface((100, 100))
    sp.gfxdraw.filled_circle(surf, 10, 10, 4, (255, 128, 0))
    assert surf.get_at((20, 20))[:3] == (255, 128, 0)
```

```
def test_draw_api_identity_at_scale_one():
    import pygame
    sp = _fresh(1.0)
    a, b = pygame.Surface((80, 80)), pygame.Surface((80, 80))
    for s, mod in ((a, sp.draw), (b, pygame.draw)):
        mod.circle(s, (10, 20, 30), (40, 40), 12, 2)
        mod.rect(s, (40, 50, 60), pygame.Rect(4, 4, 20, 10))
        mod.polygon(s, (70, 80, 90), [(1, 1), (30, 2), (20, 25)])
        mod.ellipse(s, (99, 10, 10), pygame.Rect(30, 40, 20, 12))
    assert pygame.image.tostring(a, "RGB") == pygame.image.tostring(b, "RGB")
```

☐ Step 2: 執行測試確認失敗

Run: `python3 -m pytest pixel_battle/tests/test_scaled_pygame.py -q` Expected: FAIL —
`_Draw` 沒有 `circle` 屬性

☐ Step 3: 實作

在 `scaled_pygame.py` 的 `_len` 之後加入 `_rect`, 並擴充 `_Draw`、新增 `gfxdraw`:

```
def _rect(r):
    """Scale a rect-like (pygame.Rect or a 4-tuple) into a scaled Rect."""
    x, y, w, h = r
    return _pg.Rect(round(x * S), round(y * S),
                    max(1, round(w * S)), max(1, round(h * S)))
```

`_Draw` 補上其餘方法(接在 `line` 之後):

```
@staticmethod
def lines(surface, color, closed, points, width=1):
    return _pg.draw.lines(surface, color, closed, _pts(points),
                          _len(width))

@staticmethod
def circle(surface, color, center, radius, width=0, **kwargs):
    return _pg.draw.circle(surface, color, _pt(center), _len(radius),
                          _len(width), **kwargs)

@staticmethod
def polygon(surface, color, points, width=0):
```

```

        return _pg.draw.polygon(surface, color, _pts(points), _len(width))

    @staticmethod
    def rect(surface, color, rect, width=0, **kwargs):
        return _pg.draw.rect(surface, color, _rect(rect), _len(width),
                               **kwargs)

    @staticmethod
    def ellipse(surface, color, rect, width=0):
        return _pg.draw.ellipse(surface, color, _rect(rect), _len(width))

    @staticmethod
    def arc(surface, color, rect, start_angle, stop_angle, width=1):
        return _pg.draw.arc(surface, color, _rect(rect), start_angle,
                              stop_angle, _len(width))

    @staticmethod
    def aaline(surface, color, start_pos, end_pos, blend=1):
        return _pg.draw.aaline(surface, color, _pt(start_pos), _pt(end_pos),
                                blend)

```

新增 gfxdraw 包裝(檔案尾端、`__getattr__` 之前):

```

class _GfxDraw:
    """Scaled gfxdraw. Takes ints for x/y/r rather than point tuples."""

    @staticmethod
    def aacircle(surface, x, y, r, color):
        return _pg.gfxdraw.aacircle(surface, round(x * S), round(y * S),
                                       max(1, round(r * S)), color)

    @staticmethod
    def filled_circle(surface, x, y, r, color):
        return _pg.gfxdraw.filled_circle(surface, round(x * S), round(y * S),
                                           max(1, round(r * S)), color)

```

```
gfxdraw = _GfxDraw()
```

注意: `gfxdraw` 需要 `import pygame.gfxdraw` 才會掛上 `_pg.gfxdraw`。在檔案頂端 `import pygame` as `_pg` 之後加入:

```
try:
    import pygame.gfxdraw # noqa: F401 (registers _pg.gfxdraw)
except ImportError:
    pass
```

☐ Step 4: 執行測試確認通過

Run: `python3 -m pytest pixel_battle/tests/test_scaled_pygame.py -q` Expected: PASS (11 passed)

☐ Step 5: Commit

```
git add pixel_battle/rl/scaled_pygame.py pixel_battle/tests/test_scaled_pygame.py
git commit -m "feat(pixel-battle): shim 補齊 draw API + gfxdraw (width=0 填滿語義保留)"
```

Task 4: Surface 子類 — blit / fill / subsurface

主線用量: `blit` 101、`fill` 25(其中 `impact_fx.py:872/1198/1210` 帶 `rect` 參數)、`subsurface` 2、`Surface(...)` 116。

Files: – Modify: `pixel_battle/rl/scaled_pygame.py` – Test: `pixel_battle/tests/test_scaled_pygame.py`

Interfaces: – Consumes: Task 2–3 的 `_pt` / `_rect` / `S`。 – Produces: `ScaledSurface(pygame.Surface)`、`Surface(size, flags=0, ...)` 工廠(含 CANVAS 特例)。

☐ Step 1: 寫失敗測試

Append to `pixel_battle/tests/test_scaled_pygame.py`:

```
def test_surface_size_is_scaled():
    sp = _fresh(2.0)
    s = sp.Surface((30, 40))
    assert s.get_size() == (60, 80)

def test_canvas_size_maps_to_exact_output():
```

```
"""480x854 must land on exactly 1080x1920, not round(854*2.25)=1922."""
```

```
sp = _fresh(2.25)
s = sp.Surface(sp.CANVAS)
assert s.get_size() == (1080, 1920)
```

```
def test_blit_dest_is_scaled():
    import pygame
    sp = _fresh(2.0)
    dst = sp.Surface((50, 50))
    src = pygame.Surface((4, 4))
    src.fill((255, 0, 0))
    dst.blit(src, (5, 5))
    assert dst.get_at((10, 10))[:3] == (255, 0, 0)
    assert dst.get_at((5, 5))[:3] != (255, 0, 0)
```

```
def test_blit_accepts_rect_dest():
    import pygame
    sp = _fresh(2.0)
    dst = sp.Surface((50, 50))
    src = pygame.Surface((4, 4))
    src.fill((0, 255, 0))
    dst.blit(src, pygame.Rect(5, 5, 4, 4))
    assert dst.get_at((10, 10))[:3] == (0, 255, 0)
```

```
def test_fill_with_rect_is_scaled():
    sp = _fresh(2.0)
    s = sp.Surface((50, 50))
    s.fill((0, 0, 0))
    s.fill((255, 255, 255), (5, 5, 10, 2))
    assert s.get_at((12, 12))[:3] == (255, 255, 255)
    assert s.get_at((6, 6))[:3] != (255, 255, 255)
```

```
def test_fill_without_rect_still_fills_everything():
    sp = _fresh(2.0)
    s = sp.Surface((20, 20))
```

```

s.fill((7, 8, 9))
assert s.get_at((0, 0))[:3] == (7, 8, 9)
assert s.get_at((39, 39))[:3] == (7, 8, 9)

def test_derived_surfaces_keep_scaling_behaviour():
    import pygame
    sp = _fresh(2.0)
    s = sp.Surface((40, 40))
    assert isinstance(s.copy(), sp.ScaledSurface)
    assert isinstance(s.subsurface((0, 0, 10, 10)), sp.ScaledSurface)

def test_surface_identity_at_scale_one():
    import pygame
    sp = _fresh(1.0)
    a = sp.Surface((40, 40), pygame.SRCALPHA)
    b = pygame.Surface((40, 40), pygame.SRCALPHA)
    src = pygame.Surface((6, 6)); src.fill((1, 2, 3))
    a.blit(src, (7, 9)); b.blit(src, (7, 9))
    a.fill((4, 5, 6), (2, 2, 5, 5)); b.fill((4, 5, 6), (2, 2, 5, 5))
    assert a.get_size() == b.get_size() == (40, 40)
    assert pygame.image.tostring(a, "RGBA") == pygame.image.tostring(b, "RGBA")

```

☐ Step 2: 執行測試確認失敗

Run: `python3 -m pytest pixel_battle/tests/test_scaled_pygame.py -q` Expected: FAIL — shim
目前把 `Surface` 直接轉發給真 `pygame`, 尺寸未縮放

☐ Step 3: 實作

在 `scaled_pygame.py` 中 `_rect` 之後加入:

```

class ScaledSurface(_pg.Surface):
    """A Surface whose blit/fill/subsurface arguments live in fight coords.

    Sizes are already scaled at construction (see the Surface factory), so
    these overrides only translate the *arguments* callers pass in.
    """

```



```

def blit(self, source, dest, area=None, special_flags=0):
    if dest is not None:
        if isinstance(dest, _pg.Rect):
            dest = _rect(dest)
        else:
            dest = _pt(dest)
    if area is not None:
        area = _rect(area)
    return super().blit(source, dest, area, special_flags)

def fill(self, color, rect=None, special_flags=0):
    # No rect means "the whole surface" – already scaled, leave it alone.
    if rect is not None:
        rect = _rect(rect)
    return super().fill(color, rect, special_flags)

def subsurface(self, *args):
    rect = args[0] if len(args) == 1 else args
    return super().subsurface(_rect(rect))

def Surface(size, flags=0, *args, **kwargs):
    """Create a scaled surface.

The fight canvas (480x854) maps to exactly 1080x1920 rather than the
1921.5 that 2.25x would give; the 1.5px overflow falls off the bottom,
where nothing is drawn (GROUND_Y scales to 1575).
    """
    w, h = size
    if (round(w), round(h)) == CANVAS:
        sw, sh = CANVAS_SCALED
    else:
        sw = max(1, round(w * S))
        sh = max(1, round(h * S))
    return ScaledSurface((sw, sh), flags, *args, **kwargs)

```

CANVAS_SCALED 在 $S \neq 2.25$ 時需要跟著走,否則 $S=1.0$ 的還原測試會拿到 1080x1920。把 `set_scale` 改成同時更新它:

```
def set_scale(s: float) -> None:
    """Set the global draw scale. 1.0 reproduces pre-hi-res output exactly."""
    global S, CANVAS_SCALED
    S = float(s)
    if S == 2.25:
        CANVAS_SCALED = (1080, 1920)
    else:
        CANVAS_SCALED = (max(1, round(CANVAS[0] * S)),
                          max(1, round(CANVAS[1] * S)))
```

☐ Step 4: 執行測試確認通過

Run: `python3 -m pytest pixel_battle/tests/test_scaled_pygame.py -q` Expected: PASS (19 passed)

☐ Step 5: Commit

```
git add pixel_battle/rl/scaled_pygame.py pixel_battle/tests/test_scaled_pygame.py
git commit -m "feat(pixel-battle): ScaledSurface - blit/fill/subsurface 座標縮放 + 畫布特例"
```

Task 5: Rect / font / transform

主線用量: `Rect` 3、`font.SysFont` 5、`transform.smoothscale` 6、`transform.rotate` 2。

字型注意事項: `hud.py:60-61`、`impact_fx.py:384`、`impact_fx.py:1115` 已使用 `SysFont(None, SIZE * 2)` 這種「畫大再縮」的文字超採樣手法。shim 讓它變成 `SIZE * 2 * S`, 配合同樣被縮放的 `smoothscale` 目標尺寸後仍自治 — Task 9 會逐處複核。

Files: – Modify: `pixel_battle/rl/scaled_pygame.py` – Test: `pixel_battle/tests/test_scaled_pygame.py`

Interfaces: – Consumes: Task 2–4。 – Produces: `Rect(...)`、`font.SysFont/Font/init/get_init`、`transform.smoothscale/scale/rotate/flip`。

☐ Step 1: 寫失敗測試

Append to `pixel_battle/tests/test_scaled_pygame.py`:

```

def test_rect_is_scaled():
    sp = _fresh(2.0)
    r = sp.Rect(3, 4, 10, 20)
    assert (r.x, r.y, r.w, r.h) == (6, 8, 20, 40)

def test_font_size_is_scaled():
    import pygame
    pygame.font.init()
    sp = _fresh(2.0)
    scaled = sp.font.SysFont(None, 10)      # shim turns this into a real 20pt
    plain = pygame.font.SysFont(None, 20)   # genuinely 20pt, no shim
    assert scaled.size("Ag") == plain.size("Ag")

def test_smoothscale_target_is_scaled():
    import pygame
    sp = _fresh(2.0)
    src = pygame.Surface((10, 10))
    out = sp.transform.smoothscale(src, (5, 5))
    assert out.get_size() == (10, 10)

def test_rotate_is_not_scaled():
    import pygame
    sp = _fresh(2.0)
    src = pygame.Surface((10, 20))
    out = sp.transform.rotate(src, 90)
    assert out.get_size() == pygame.transform.rotate(src, 90).get_size()

```

☐ Step 2: 執行測試確認失敗

Run: `python3 -m pytest pixel_battle/tests/test_scaled_pygame.py -q` Expected: FAIL —
 Rect 未縮放(轉發給真 pygame)

☐ Step 3: 實作

在 `scaled_pygame.py` 加入(置於 `Surface` 工廠之後):

```

def Rect(*args):
    """Create a scaled Rect from fight coordinates."""
    if len(args) == 1:
        return _rect(args[0])
    if len(args) == 2:      # (pos, size)
        (x, y), (w, h) = args
        return _rect((x, y, w, h))
    return _rect(args)


class _Font:
    """Font factory with scaled point sizes."""

    @staticmethod
    def SysFont(name, size, bold=False, italic=False):
        return _pg.font.SysFont(name, max(1, round(size * S)), bold, italic)

    @staticmethod
    def Font(name, size):
        return _pg.font.Font(name, max(1, round(size * S)))

    @staticmethod
    def init():
        return _pg.font.init()

    @staticmethod
    def get_init():
        return _pg.font.get_init()

    @staticmethod
    def get_default_font():
        return _pg.font.get_default_font()


font = _Font()


class _Transform:

```

```
"""Transforms. Target SIZES are in fight coords; angles are not."""
```

```
@staticmethod
```

```
def smoothscale(surface, size, dest_surface=None):  
    sz = (max(1, round(size[0] * S)), max(1, round(size[1] * S)))  
    if dest_surface is not None:  
        return _pg.transform.smoothscale(surface, sz, dest_surface)  
    return _pg.transform.smoothscale(surface, sz)
```

```
@staticmethod
```

```
def scale(surface, size, dest_surface=None):  
    sz = (max(1, round(size[0] * S)), max(1, round(size[1] * S)))  
    if dest_surface is not None:  
        return _pg.transform.scale(surface, sz, dest_surface)  
    return _pg.transform.scale(surface, sz)
```

```
@staticmethod
```

```
def rotate(surface, angle):  
    return _pg.transform.rotate(surface, angle)
```

```
@staticmethod
```

```
def flip(surface, flip_x, flip_y):  
    return _pg.transform.flip(surface, flip_x, flip_y)
```

```
@staticmethod
```

```
def rotozoom(surface, angle, scale):  
    return _pg.transform.rotozoom(surface, angle, scale)
```

```
transform = _Transform()
```

☐ Step 4: 執行測試確認通過

Run: `python3 -m pytest pixel_battle/tests/test_scaled_pygame.py -q` Expected: PASS (23 passed)

☐ Step 5: Commit

```
git add pixel_battle/rl/scaled_pygame.py pixel_battle/tests/test_scaled_pygame.py
git commit -m "feat(pixel-battle): shim 補上 Rect / font / transform 縮放"
```

Task 6: 驗證工具 — events 比對與 SSIM

在掛上 shim 之前先備妥,掛上後才能立刻驗。

Files: – Create: `tools/hires_verify.py` – Test: `pixel_battle/tests/test_hires_verify.py`

Interfaces: – Consumes: Task 1 的 `events.json` 與 `frame_*.png` 格式。 – Produces:
`compare_events(baseline_json, current_json) -> list[str]` (差異描述,空 list 代表相同)、
`frame_ssim(baseline_png, current_png) -> float` (把 current 降回 baseline 尺寸後比對)。

☐ Step 1: 寫失敗測試

Create `pixel_battle/tests/test_hires_verify.py`:

```
"""hi-res verification helpers – event equality and structural similarity."""
import json
import os
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")

def _write(tmp_path, name, payload):
    p = tmp_path / name
    p.write_text(json.dumps(payload), encoding="utf-8")
    return p

def test_identical_events_report_no_differences(tmp_path):
    from tools.hires_verify import compare_events
    data = {"winner": "left", "n_frames": 10, "events": [{"t": 1, "kind": "hit"}],
           "event_video_ms": {"0": 16}}
    a = _write(tmp_path, "a.json", data)
    b = _write(tmp_path, "b.json", data)
    assert compare_events(a, b) == []

def test_changed_winner_is_reported(tmp_path):
```

```

from tools.hires_verify import compare_events
base = {"winner": "left", "n_frames": 10, "events": [], "event_video_ms": {}}
cur = dict(base, winner="right")
diffs = compare_events(_write(tmp_path, "a.json", base),
                       _write(tmp_path, "b.json", cur))
assert any("winner" in d for d in diffs)

```

```

def test_changed_event_count_is_reported(tmp_path):
    from tools.hires_verify import compare_events
    base = {"winner": "left", "n_frames": 10,
            "events": [{"t": 1}], "event_video_ms": {}}
    cur = dict(base, events=[{"t": 1}, {"t": 2}])
    diffs = compare_events(_write(tmp_path, "a.json", base),
                          _write(tmp_path, "b.json", cur))
    assert any("event" in d.lower() for d in diffs)

```

```

def test_ssim_of_a_scaled_copy_is_high(tmp_path):
    """A faithful 2x render, downscaled back, must match the original."""
    import numpy as np
    from PIL import Image, ImageDraw
    from tools.hires_verify import frame_ssim

    base = Image.new("RGB", (120, 200), (20, 20, 30))
    d = ImageDraw.Draw(base)
    d.line((10, 10, 100, 180), fill=(240, 240, 240), width=3)
    d.ellipse((40, 60, 80, 100), fill=(200, 40, 40))
    bp = tmp_path / "base.png"; base.save(bp)

    big = base.resize((240, 400), Image.LANCZOS)
    cp = tmp_path / "cur.png"; big.save(cp)

    assert frame_ssim(bp, cp) > 0.90

```

```

def test_ssim_drops_when_an_element_moves(tmp_path):
    from PIL import Image, ImageDraw
    from tools.hires_verify import frame_ssim

```

```

base = Image.new("RGB", (120, 200), (20, 20, 30))
ImageDraw.Draw(base).ellipse((40, 60, 80, 100), fill=(200, 40, 40))
bp = tmp_path / "base.png"; base.save(bp)

moved = Image.new("RGB", (240, 400), (20, 20, 30))
# same circle but shifted – this is what a missed *S looks like
ImageDraw.Draw(moved).ellipse((80, 260, 160, 340), fill=(200, 40, 40))
cp = tmp_path / "cur.png"; moved.save(cp)

assert frame_ssim(bp, cp) < 0.90

```

☐ Step 2: 執行測試確認失敗

Run: `python3 -m pytest pixel_battle/tests/test_hires_verify.py -q` Expected: FAIL —
 ModuleNotFoundError: No module named 'tools.hires_verify'

☐ Step 3: 實作

Create `tools/__init__.py` (空檔,讓 `tools.hires_verify` 可被 import):

```
touch tools/__init__.py
```

Create `tools/hires_verify.py`:

```

"""Verify a hi-res render against a pre-change baseline.

Two independent checks:
    compare_events – the fight itself must be untouched (same events, same
                     timings, same winner). Physics is not scaled, so any
                     difference here means the shim leaked into game logic.
    frame_ssim – the new frame, downscaled back to baseline size, must
                  match the baseline. This is what catches an element
                  whose coordinates were never multiplied by S.
"""

from __future__ import annotations

import json
from pathlib import Path

```



```

import numpy as np
from PIL import Image
from skimage.metrics import structural_similarity

def _load(p) -> dict:
    return json.loads(Path(p).read_text(encoding="utf-8"))

def compare_events(baseline_json, current_json) -> list[str]:
    """Return human-readable differences. Empty list means identical."""
    a, b = _load(baseline_json), _load(current_json)
    diffs: list[str] = []

    for key in ("winner", "terminated", "n_frames"):
        if a.get(key) != b.get(key):
            diffs.append(f"{key}: baseline={a.get(key)!r} current={b.get(key)!r}")

    ea, eb = a.get("events", []), b.get("events", [])
    if len(ea) != len(eb):
        diffs.append(f"event count: baseline={len(ea)} current={len(eb)}")
    else:
        for i, (x, y) in enumerate(zip(ea, eb)):
            if x != y:
                diffs.append(f"event[{i}]: baseline={x!r} current={y!r}")

    ma, mb = a.get("event_video_ms", {}), b.get("event_video_ms", {})
    if ma != mb:
        diffs.append("event_video_ms differs")

    return diffs

def frame_ssim(baseline_png, current_png) -> float:
    """SSIM after downscaling `current` to the baseline's dimensions."""
    base = Image.open(baseline_png).convert("RGB")
    cur = Image.open(current_png).convert("RGB")
    if cur.size != base.size:
        cur = cur.resize(base.size, Image.LANCZOS)

```

```

a = np.asarray(base, dtype=np.float64)
b = np.asarray(cur, dtype=np.float64)
return float(structural_similarity(a, b, channel_axis=2, data_range=255.0))

def report(baseline_dir, current_dir, ssim_floor: float = 0.90) -> int:
    """Print a full comparison. Returns the number of problems found."""
    baseline_dir, current_dir = Path(baseline_dir), Path(current_dir)
    problems = 0

    diffs = compare_events(baseline_dir / "events.json",
                           current_dir / "events.json")

    if diffs:
        problems += len(diffs)
        print("FIGHT BEHAVIOUR CHANGED – the shim leaked into game logic:")
        for d in diffs:
            print("  -", d)
    else:
        print("fight behaviour: identical")

    print("\nframe structure (SSIM after downscale):")
    for bp in sorted(baseline_dir.glob("frame_*.png")):
        cp = current_dir / bp.name
        if not cp.exists():
            print(f"  {bp.name}: MISSING in current")
            problems += 1
            continue
        s = frame_ssim(bp, cp)
        flag = "ok" if s >= ssim_floor else "LOW"
        if s < ssim_floor:
            problems += 1
        print(f"  {bp.name}: {s:.4f} {flag}")

    print(f"\nproblems: {problems}")
    return problems

if __name__ == "__main__":
    import sys

```

```

root = Path(__file__).resolve().parents[1]
b = Path(sys.argv[1]) if len(sys.argv) > 1 else \
    root / "pixel_battle/output/hires_baseline/b01"
c = Path(sys.argv[2]) if len(sys.argv) > 2 else \
    root / "pixel_battle/output/hires_current/b01"
raise SystemExit(1 if report(b, c) else 0)

```

☐ Step 4: 執行測試確認通過

Run: `python3 -m pytest pixel_battle/tests/test_hires_verify.py -q` Expected: PASS (5 passed)

☐ Step 5: Commit

```

git add tools/__init__.py tools/hires_verify.py
    pixel_battle/tests/test_hires_verify.py
git commit -m "test(pixel-battle): hi-res 驗證工具 - events 比對 + 降採樣 SSIM"

```

Task 7: 掛載 shim 與錄影尺寸

Files: – Modify: `pixel_battle/rl/stick_renderer.py:11` (與 line 14 的 `gfxdraw try` 區塊) – Modify: `pixel_battle/rl/impact_fx.py` (import `pygame` 行) – Modify: `pixel_battle/rl/hud.py` (import `pygame` 行) – Modify: `pixel_battle/rl/weapons.py` (import `pygame` 行) – Modify: `pixel_battle/rl/play.py` (import `pygame` 行、2471、2519) – Modify: `pixel_battle/rl/play_scripted.py:74` (僅 recorder 尺寸,不套 shim)

Interfaces: – Consumes: Task 2–5 的 shim。 – Produces: 主線渲染輸出 1080x1920。

注意 `play_scripted.py` 不套 shim: 它的 `pygame` 只用於 `pygame.init()` 與 `pygame.display.set_mode((1, 1))` (line 35–36),與繪圖無關。

☐ Step 1: 替換 5 個渲染模組的 import

每個檔案把 `import pygame` 換成下面兩行(保留原有的 `# noqa` 註記):

```

# Renders at CANVAS_SCALED; see scaled_pygame's docstring. NOT real pygame.
from pixel_battle.rl import scaled_pygame as pygame

```

`stick_renderer.py` 另有 line 12–17 的 `gfxdraw try` 區塊,改為:

```
try:
    _gfxdraw = pygame.gfxdraw
    _HAS_GFXDRAW = True
except AttributeError:
    _HAS_GFXDRAW = False
```

Run: `grep -n "scaled_pygame" pixel_battle/rl/stick_renderer.py`
`pixel_battle/rl/impact_fx.py pixel_battle/rl/hud.py pixel_battle/rl/weapons.py`
`pixel_battle/rl/play.py` Expected: 5 個檔案各出現一次

☐ Step 2: 確認物理層沒有被波及

Run: `grep -rn "scaled_pygame" pixel_battle/engine/` Expected: 無輸出(engine/ 完全不碰 shim)

☐ Step 3: 調整 FrameRecorder 尺寸

`play.py:2471` 與 `play.py:2519`,以及 `play_scripted.py:74`,把 `width=WIDTH, height=HEIGHT` 改為畫布尺寸。

`play.py` 內(shim 已為該檔的 `pygame`):

```
recorder = FrameRecorder(str(raw_video), fps=RENDER_FPS,
                          width=pygame.CANVAS_SCALED[0],
                          height=pygame.CANVAS_SCALED[1])
```

`play_scripted.py` 沒有套 shim,需顯式 import(加在既有 import 區塊):

```
from pixel_battle.rl import scaled_pygame as _scaled # noqa: E402
```

然後 line 74 改為:

```
recorder = FrameRecorder(str(raw), fps=RENDER_FPS,
                          width=_scaled.CANVAS_SCALED[0],
                          height=_scaled.CANVAS_SCALED[1])
```

☐ Step 4: 冒煙測試 — 渲染是否還跑得動

Run: `python3 -m pixel_battle.rl.play_scripted`
`pixel_battle/data/scripts/b01_lumen_jugg.yaml` Expected: 印出 `Scripted render:`
`.../b01_lumen_jugg_raw.mp4`,無例外

☐ **Step 5: 確認輸出尺寸**

Run: `ffprobe -v error -select_streams v:0 -show_entries stream=width,height -of csv=p=0 pixel_battle/output/scripted/b01_lumen_jugg_raw.mp4` Expected: `1080,1920`

☐ **Step 6: 跑既有測試套件**

Run: `python3 -m pytest -q --no-header 2>&1 | tail -8` Expected: `555 passed` 加上本計畫新增的測試;失敗數仍為 5,且是 Global Constraints 列出的那 5 個。若出現第 6 個失敗,停下來修。

☐ **Step 7: Commit**

```
git add pixel_battle/rl/stick_renderer.py pixel_battle/rl/impact_fx.py \  
      pixel_battle/rl/hud.py pixel_battle/rl/weapons.py \  
      pixel_battle/rl/play.py pixel_battle/rl/play_scripted.py  
git commit -m "feat(pixel-battle): 主線渲染掛上 scaled_pygame - 原生 1080x1920"
```

Task 8: 首次全套驗證,產出落差清單

Files: – Create: `pixel_battle/output/hires_current/` (產物,已被 Task 1 的 `.gitignore` 規則涵蓋前需確認) – Modify: `.gitignore` (若尚未涵蓋)

Interfaces: – Consumes: Task 1 基準、Task 6 驗證工具、Task 7 掛載結果。 – Produces: 落差清單,供 Task 9 修正。

☐ **Step 1: 確認 current 產物也被 gitignore 涵蓋**

`.gitignore` 中應有(Task 1 已加入第一條,若沒有第二條就補上):

```
pixel_battle/output/hires_baseline/  
pixel_battle/output/hires_current/
```

☐ **Step 2: 用現行(S=2.25)渲染產生兩組對照**

Run:

```
python3 tools/hires_baseline.py \  
  pixel_battle/data/scripts/b01_lumen_jugg.yaml \  
  pixel_battle/output/hires_current/b01  
python3 tools/hires_baseline.py \  
  pixel_battle/data/scripts/b01_lumen_jugg.yaml \  
  pixel_battle/output/hires_baseline/b01
```

```
pixel_battle/data/scripts/b12_shade_deadeye.yaml \
pixel_battle/output/hires_current/b12
```

Expected: frame 數與 winner 與各自基準相同, canvas=[1080, 1920]

☐ **Step 3: 確認對照組幀為 1080x1920**

Run:

```
python3 -c "  
from PIL import Image  
for b in ('b01', 'b12'):  
    print(b,  
          Image.open(f'pixel_battle/output/hires_current/{b}/frame_00600.png').size)  
"
```

Expected: 兩行都是 (1080, 1920)

☐ **Step 4: 執行比對**

Run:

```
python3 tools/hires_verify.py \  
pixel_battle/output/hires_baseline/b01 pixel_battle/output/hires_current/b01  
python3 tools/hires_verify.py \  
pixel_battle/output/hires_baseline/b12 pixel_battle/output/hires_current/b12
```

Expected: 兩組第一層都是 `fight behaviour: identical`。第二層可能出現若干 `LOW` — 那正是漏乘 `S` 的元素,記下是哪一部、哪一幀、SSIM 值,交給 Task 9。

若第一層就報 `FIGHT BEHAVIOUR CHANGED`,立即停止並排查 shim 是否洩漏進 `engine/` (重跑 Task 7 Step 2)。

☐ **Step 5: 把落差清單寫進 commit message 留痕**

```
git commit --allow-empty -m "chore(pixel-battle): hi-res 首次驗證結果  
  
fight behaviour: identical  
frame SSIM: <逐一貼上 tools/hires_verify.py 的輸出>"
```

Task 9: 系統審查 52 處尺寸查詢與風險點,修至驗證全綠

Task 8 的 SSIM 落差指出症狀,本任務做系統性排查。

Files: – Modify: `pixel_battle/rl/impact_fx.py` (get_size 類 21 處) – Modify: `pixel_battle/rl/play.py` (get_size 類 21 處、398 的 surfarray) – Modify: `pixel_battle/rl/stick_renderer.py` (5 處) – Modify: `pixel_battle/rl/hud.py` (4 處) – Modify: `pixel_battle/rl/weapons.py` (1 處)

Interfaces: – Consumes: Task 8 的落差清單。 – Produces: `tools/hires_verify.py` 全綠 (problems: 0)。

☐ Step 1: 列出所有待審查點

Run:

```
grep -rnE "get_size\(\)|get_rect\(\)|get_width\(\)|get_height\(\)" \  
pixel_battle/rl/hud.py pixel_battle/rl/impact_fx.py pixel_battle/rl/play.py \  
pixel_battle/rl/stick_renderer.py pixel_battle/rl/weapons.py
```

Expected: 52 行

☐ Step 2: 逐處判定

判定準則: – 相對用法(例如 `w // 2` 置中、`(0, 0, *surf.get_size())` 覆蓋全表面)– 尺寸已縮放,計算自動正確,不需改。 – 絕對像素偏移(例如 `surf.get_width() - 40` 的 40 是未縮放常數)– 混用了縮放與未縮放的量,需把常數改為 `40 * pygame.S` 或改用比例。 – 把判定結果寫成註解或記在 commit message,便於日後追溯。

☐ Step 3: 檢查 surfarray 用法

Run: `sed -n '390,410p' pixel_battle/rl/play.py`

`play.py:398` 的 `import pygame.surfarray as _sa` 直接操作像素緩衝。確認其陣列尺寸來自 `surface.get_size()` (自動正確)而非硬編 480/854(需改)。

☐ Step 4: 複核字型二倍技巧

Run: `grep -n "SysFont" pixel_battle/rl/hud.py pixel_battle/rl/impact_fx.py`

確認 `hud.py:60-61`、`impact_fx.py:384`、`impact_fx.py:1115` 的 `SIZE * 2` 與其後的 smoothscale 目標尺寸在縮放後仍成比例。文字若過大或模糊,調整該處的縮小目標。

☐ Step 5: 重新驗證兩組

Run:

```
python3 tools/hires_baseline.py \  
  pixel_battle/data/scripts/b01_lumen_jugg.yaml \  
  pixel_battle/output/hires_current/b01  
python3 tools/hires_baseline.py \  
  pixel_battle/data/scripts/b12_shade_deadeye.yaml \  
  pixel_battle/output/hires_current/b12  
python3 tools/hires_verify.py \  
  pixel_battle/output/hires_baseline/b01 pixel_battle/output/hires_current/b01  
python3 tools/hires_verify.py \  
  pixel_battle/output/hires_baseline/b12 pixel_battle/output/hires_current/b12
```

Expected: 兩組都是 `fight behaviour: identical`、所有幀 `ok`、`problems: 0`

b12 是遠距對峙,會走到投射物與拉開鏡頭的繪圖路徑;b01 全綠而 b12 不綠, 代表漏縮放集中在那些路徑上。

☐ Step 6: 修到兩組全綠才算完成

若仍有 `LOW`,回到 Step 2 的判定準則重新檢視該幀對應的繪圖路徑。用 `Read` 工具直接看 baseline 與 current 的同一幀 PNG,肉眼定位是哪個元素 跑位 — SSIM 只告訴你有問題,看圖才知道是誰。

☐ Step 7: 跑完整測試套件

Run: `python3 -m pytest -q --no-header 2>&1 | tail -6` Expected: 失敗數仍為 5(Global Constraints 列出的那 5 個)

☐ Step 8: Commit

```
git add -A pixel_battle/rl/  
git commit -m "fix(pixel-battle): 修正 hi-res 下混用縮放與未縮放量的座標計算  
  
<列出實際修改的檔案與行號,以及每處的判定理由>"
```

Task 10: 移除 build_short.py 的上採樣補償

Files: – Modify: `pixel_battle/scripts/build_short.py:86` 附近的濾鏡鏈

Interfaces: – Consumes: Task 7–9 產出的原生 1080x1920 raw 。 – Produces: 不再重複縮放與過銳的成品。

☐ **Step 1: 檢視現行濾鏡鏈**

Run: `sed -n '80,96p' pixel_battle/scripts/build_short.py`

☐ **Step 2: 移除 scale 與 unsharp**

把濾鏡鏈中的這兩行刪除:

```
f"scale={W}:{H}:flags=lanczos,"  
f"unsharp=5:5:0.35:5:5:0.0,"
```

保留 `gradfun=1.2:16`, 與 `noise=alls=2:allf=t` 。刪除後 `[0:v]trim=...` 之後應直接接 `gradfun` 。

同時更新檔案頂端 docstring 第 6–7 行,原文說 “upscales 480x854 -> 1080x1920”,改為說明 raw 已是原生 1080x1920 。

☐ **Step 3: 建構一支成品**

Run:

```
python3 -m pixel_battle.scripts.build_short \  
pixel_battle/output/scripted/b01_lumen_jugg_raw.mp4 \  
pixel_battle/output/shorts/b01_lumen_jugg.mp4 \  
"誰會贏?" "LUMEN VS JUGGERNAUT" "LUMEN WINS" 1.5
```

Expected: 成功產出檔案,無 ffmpeg 錯誤

☐ **Step 4: 確認成品尺寸與時長**

Run:

```
ffprobe -v error -select_streams v:0 \  
-show_entries stream=width,height,r_frame_rate,duration \  
-of csv=p=0 pixel_battle/output/shorts/b01_lumen_jugg.mp4
```

Expected: `1080,1920,30/1,<約 53 秒>`

☐ **Step 5: Commit**

```
git add pixel_battle/scripts/build_short.py
git commit -m "fix(pixel-battle): build_short 移除上採樣與補償銳化 - 來源已是原生 1080x1920"
```

Task 11: 重新渲染 b01–b21 並視覺驗收

Files: – Create: `pixel_battle/output/scripted/*_raw.mp4` (21 支) – Create: `pixel_battle/output/shorts/*.mp4` (21 支)

Interfaces: – Consumes: Task 7–10 的完整管線。 – Produces: 補回遺失的成品庫存。

☐ Step 1: 確認 21 份腳本齊全

Run: `ls pixel_battle/data/scripts/b*.yaml | wc -l` Expected: 21

☐ Step 2: 寫批次腳本(同時渲染並記錄勝者)

`build_short` 需要三個文案參數,其中勝者只有渲染後才知道。舊的 `output/shorts/POSTING.md` 清單已隨成品一起遺失且本就 `gitignored`, 因此這裡重建一份 `manifest.json` 取代它。

Create `tools/build_shorts_batch.py`:

```
"""Render every b*.yaml and build a posting-ready short for each.

Writes output/shorts/manifest.json recording matchup and winner – this
replaces the old POSTING.md, which was gitignored and lost with the
previous render batch.
"""

from __future__ import annotations
import json
import os
import subprocess
import sys
from pathlib import Path

os.environ.setdefault("SDL_VIDEODRIVER", "dummy")

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))
```

```
SCRIPTS = ROOT / "pixel_battle/data/scripts"
RAW_DIR = ROOT / "pixel_battle/output/scripted"
OUT_DIR = ROOT / "pixel_battle/output/shorts"
CUT_START = "1.5"
```

```
def _display_names() -> dict:
    data = json.loads((ROOT / "pixel_battle/data/characters.json")
                      .read_text(encoding="utf-8"))
    return {k: v.get("display_name", k) for k, v in data.items()}
```

```
def render_one(script_path: Path) -> dict:
    """Render one fight, returning its matchup and winner."""
    import yaml
    import pixel_battle.rl.play_scripted as ps

    captured: dict = {}
    orig = ps._render_fight

    def _tap(*a, **k):
        r = orig(*a, **k)
        captured.update(r)
        return r

    ps._render_fight = _tap
    try:
        raw = ps.render_script(script_path)
    finally:
        ps._render_fight = orig

    meta = yaml.safe_load(script_path.read_text(encoding="utf-8"))
    return {
        "script": script_path.name,
        "raw": str(raw),
        "left": meta.get("left"),
        "right": meta.get("right"),
        "winner": captured.get("winner"),
    }
```

```
}
```

```
def build_one(entry: dict, names: dict) -> Path:
    left = names.get(entry["left"], entry["left"]).upper()
    right = names.get(entry["right"], entry["right"]).upper()
    win_id = entry["left"] if entry["winner"] == "left" else entry["right"]
    win = names.get(win_id, win_id).upper()

    OUT_DIR.mkdir(parents=True, exist_ok=True)
    out = OUT_DIR / (Path(entry["script"]).stem + ".mp4")
    subprocess.run([
        sys.executable, "-m", "pixel_battle.scripts.build_short",
        entry["raw"], str(out),
        "誰會贏?", f"{left} VS {right}", f"{win} WINS", CUT_START,
    ], check=True, cwd=str(ROOT))
    entry["short"] = str(out)
    entry["title"] = f"{left} VS {right}"
    return out


if __name__ == "__main__":
    names = _display_names()
    manifest = []
    for script in sorted(SCRIPTS.glob("b*.yaml")):
        print(f"=== {script.name} ===", flush=True)
        try:
            entry = render_one(script)
            build_one(entry, names)
            manifest.append(entry)
            print(f"    {entry['title']} -> winner={entry['winner']}")
        except Exception as e:
            print(f"    FAILED: {type(e).__name__}: {e}")
            manifest.append({"script": script.name, "error": str(e)})
    (OUT_DIR / "manifest.json").write_text(
        json.dumps(manifest, indent=2, ensure_ascii=False), encoding="utf-8")
    ok = sum(1 for m in manifest if "error" not in m)
    print(f"\nbuilt {ok}/{len(manifest)}")
```

☐ Step 3: 執行批次

hi-res 下單部預估 2-3 分鐘,21 部連同 build_short 約 60-90 分鐘。

Run: `python3 tools/build_shorts_batch.py 2>&1 | tail -30` Expected: 最後一行 `built 21/21`, 無 FAILED

☐ Step 4: 確認 21 支 raw 與成品都是 1080x1920

Run:

```
for f in pixel_battle/output/scripted/b*_raw.mp4 pixel_battle/output/shorts/b*_mp4;
do
    echo "$(basename $f) $(ffprobe -v error -select_streams v:0 \
        -show_entries stream=width,height -of csv=p=0 "$f")"
done
```

Expected: 全部 `1080,1920`

☐ Step 5: 視覺驗收

REQUIRED SUB-SKILL: 使用 `visual-acceptance` skill 逐項自評 — 這是把影片交給 arlong 之前的硬性關卡,沒逐項過關就先修,不准說「做好了」。

至少抽三部(近戰 b01、遠距 b12、有大招的一部),各抽 3-4 幀存成 PNG, 並用 `Read` 工具實際看過:

```
ffmpeg -y -i pixel_battle/output/shorts/b01_lumen_jugg.mp4 \
    -vf "select='eq(n\,120)+eq(n\,900)+eq(n\,1450)'" -vsync 0 \
    /tmp/b01_check_%02d.png
```

逐項確認: 線條銳利無糊邊、火柴人四肢比例正常、HUD 文字清晰不爆框、特效與武器位置正確、地板線高度與舊版一致、hook/winner 字卡未被畫面元素遮擋。

與舊版對照: `git show` 無法取回已遺失的舊成品,改用 Task 1 產生的

`pixel_battle/output/hires_baseline/b01/frame_*.png` (480x854)作為構圖 參考,確認元素相對位置一致。

☐ Step 6: 最終測試套件

Run: `python3 -m pytest -q --no-header 2>&1 | tail -6` Expected: 失敗數仍為 5

☐ Step 7: Commit

輸出目錄已被 gitignore,本次 commit 僅記錄里程碑:

```
git commit --allow-empty -m "chore(pixel-battle): b01-b21 以原生 1080x1920 重新渲染完成
```

<貼上 21 支的尺寸確認輸出摘要>"

完成標準

- `python3 tools/hires_verify.py` 回報 `fight behaviour: identical` 且 `problems: 0`
- 21 支 raw 全為 1080x1920
- 測試套件失敗數仍為 5,且是 Global Constraints 所列的 5 個
- 視覺驗收通過(visual-acceptance skill 逐項)